다이스 POS 기말 프로젝트 — 개인 개발 문서

⑥ 김하진 (2025105269)

손님 주문받기(POS) · 키오스크 주문 · 이용 현황

이 문서는 **설계 문서**입니다. 무엇을, 어떤 컴포넌트·이벤트·DB 작업으로 만들지 정해 두었고 Java 코드는 넣지 않았습니다. 코드는 직접 작성해야 결과물 테스트의 **이해도 평가(10 점)**에서 설명할 수 있습니다. 공통 클래스·DB 생성 SQL·규칙은 **「공통 개발 가이드」**를 함께 보세요. 화면 그림은 배치 참고용이며, 실제 구현은 9 주차 예시처럼 **기본 Swing 모양**이면 충분합니다.

1. 내 담당 한눈에 보기

음식·음료 **주문**을 맡는다. 주문은 헤더(orders) + 상세(order_detail) 구조로, 한 번 주문하면 orders 1 줄, 메뉴마다 order_detail 1 줄을 저장한다. 키오스크 **이용 현황** 화면도 맡는다(③ 에서 옮겨 옴).

담당 화면	구분	비고
손님 주문받기	직원	근무자 확인 후 열림
키오스크 음식·음료 주문 (+ 주문 확인 팝업)	키오스크	손님이 쓰는 키오스크 창
이용 현황	키오스크	손님이 쓰는 키오스크 창

담당 테이블	orders (주문), order_detail (주문 상세)
참조 테이블	menu (메뉴), table_usage (테이블 이용)
공통 클래스 (내가 만듦)	없음 (다른 팀원의 공통 클래스를 사용)

2. 평가표 대응 — 내 화면에서 보여 줄 것

결과물 테스트는 **팀 개발이지만 자기가 개발한 영역만** 평가합니다(40 점 + 보너스 5 점). 항목마다 내 화면에서 보여 줄 것을 정리했습니다.

평가 항목 (배점)	내가 보여 줄 것
사용된 컴포넌트 다양성 (3)	메뉴 JButton 동적 생성, 직접 만든 MenuCard(JPanel), JList(장바구니) + 셀 렌더러, JTable, JComboBox(테이블), JTabbedPane(주문정보·이용정보), 숫자 키패드(JButton 12 개), JDialog, JLabel
컨테이너·배치 다양성 (3)	GridLayout(메뉴 버튼·카드·키패드), BorderLayout, JTabbedPane, JScrollPane, FlowLayout(분류 탭)
다양한 이벤트 처리 (4)	리스너 하나를 모든 메뉴 버튼에 등록(getActionCommand), ItemListener(테이블 선택), KeyListener(실제 키보드 숫자), ListSelectionListener, MouseListener(메뉴 카드), javax.swing.Timer(5 초 알림), Thread(30 초 이용 현황)
DB 삽입·수정·삭제·조회 (5)	orders·order_detail C·R·U·D 모두 (아래 CRUD 표)
계획서대로 완성 (5)	수정본 3.2 ⑥ 기능 표 10 줄 + 저장 예시가 기준
이해도 1·2 (10)	8 장 예상 질문 — 특히 “주문을 왜 2 개 테이블로 나눴는지”, “리스너 하나로 메뉴 버튼 전부 처리”
전체 완성 기여도 (5)	② 결제가 쓰는 주문 상세 제공, ④ 메뉴 삭제 확인 메서드 제공, ③ 키오스크 창에 화면 2 개 연결
난이도 (5)	주문 트랜잭션(헤더 + 상세 + 재고), 버튼·카드 동적 생성, 타이머·스레드 두 가지 갱신
보너스 아이디어 (선택)	키오스크 주문에 요청사항 메모 / 인기 메뉴에 ‘BEST’ 표시

이벤트 종류 목록 (“이벤트 종류가 얼마나 많은가”)

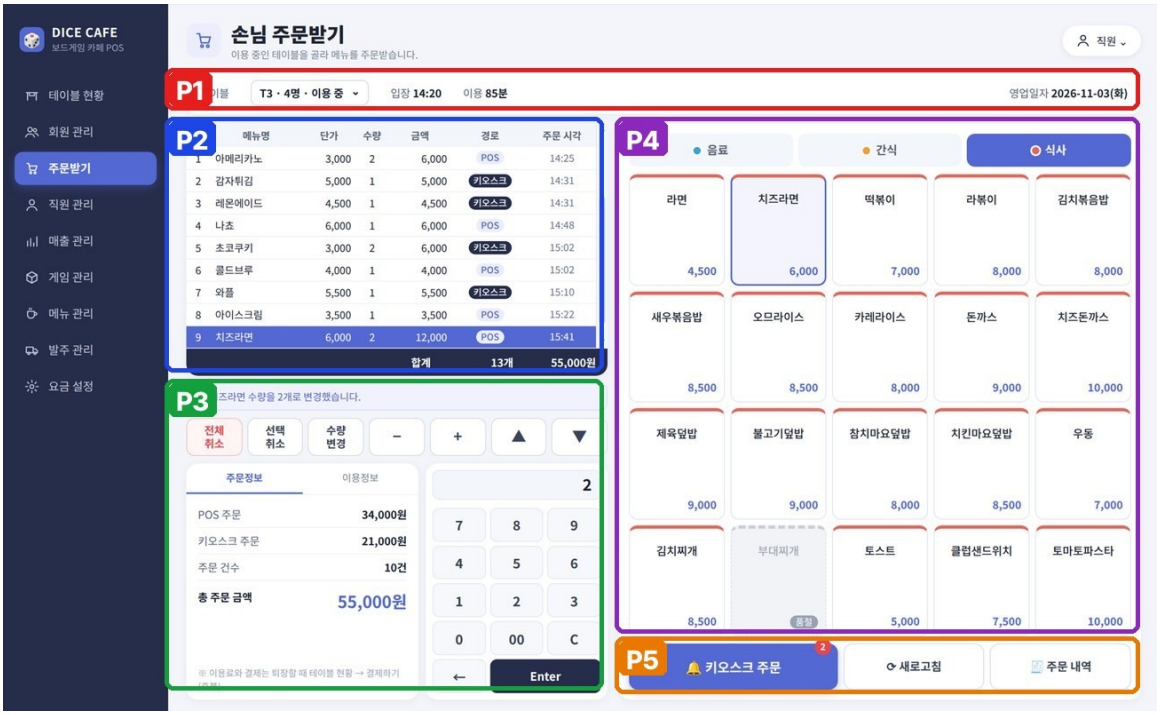
리스너 / 방식	쓰는 곳
ActionListener (하나를 공유)	모든 메뉴 버튼에 같은 리스너 객체를 등록하고 getActionCommand()로 어떤 메뉴인지 구분
ActionListener	키패드, [주문(Enter)][선택 취소][전체 취소][+][−][주문하기][다시 고르기]
ItemListener	테이블 JComboBox 선택 → 주문 내역 표시
KeyListener	실제 키보드 숫자·Enter 로 수량 입력과 주문
ListSelectionListener	주문 내역 행 선택 → [선택 취소]·[수량 변경] 켜기
MouseListener	키오스크 메뉴 카드 클릭 → 장바구니
javax.swing.Timer	5 초마다 키오스크 새 주문 수 확인
Thread	30 초마다 이용 현황 다시 계산

내 테이블의 등록·조회·수정·삭제 (DB 연동 5 점)

테이블	등록 (C)	조회 (R)	수정 (U)	삭제 (D)
orders	[주문] (POS), [주문하기] (키오스크)	주문 내역, 새 키오스크 주문 수	합계 다시 계산, ‘접수’ → ‘확인’	[전체 취소], 상세가 없어진 주문
order_detail	주문할 때 메뉴마다	주문 내역, 이용 현황	[+][−][수량 변경]	[선택 취소]

3. 화면 명세 (9 주차 예시 방식: P 영역 → 동작 → 버튼 규칙 → 경고창)

3.1 손님 주문받기 [직원]



[그림 ⑥-1] 손님 주문받기 — 색 상자가 P 영역

가. 화면 영역 구성

영역	컴포넌트	설명
P1	JComboBox(테이블: 이용 중만) + JLabel(입장 시각·이용 시간)	
P2	JTable(주문 내역)	메뉴, 단가, 수량, 금액, 경로(POS/키오스크), 시각. 아직 저장 안 한 '대기' 줄은 노란색
P3	JButton [전체 취소][선택 취소][수량 변경][−][+], JTabbedPane(주문정보/이용정보), 숫자 키패드(GridLayout 4×3) + [Enter]	Enter = 대기 줄 주문 저장
P4	분류 JToggleButton + 메뉴 JButton 들(GridLayout, 동적 생성)	품절은 꺼짐
P5	JButton [키오스크 주문 (숫자)] [새로고침] [주문 내역]	

나. 이벤트별 동작

이벤트 (리스너)	동작	DB
테이블 선택 (ItemListener)	그 이용 건의 orders·order_detail JOIN 조회 → P2	R orders, order_detail
분류 선택	④ MenuDAO.findByCategory() → 메뉴 버튼 다시 만들기	R menu
메뉴 버튼 (공유 ActionListener)	대기 줄에 담기(같은 메뉴면 수량 +1)	—
키패드 / 키보드 숫자 (KeyListener)	고른 줄의 수량 입력	—
[Enter] (주문)	주문 저장 트랜잭션(개인 문서 4.3), 경로 'POS', 상태 '확인'	C orders, order_detail
[+][−][수량 변경]	저장된 줄이면 order_detail qty·subtotal UPDATE + orders.total_amount 다시 계산 + 재고 조정	U order_detail, orders

이벤트 (리스너)	동작	DB
[선택 취소] / [전체 취소]	확인창 → 주문 취소 트랜잭션	D order_detail, orders
5 초마다 (Timer)	경로 '키오스크' AND 상태 '접수' 수 → P5 버튼 숫자. 누르면 목록 보여 주고 '확인'으로 UPDATE	R / U orders

다. 버튼 켜기·끄기 규칙

상황	규칙
테이블을 고르지 않았을 때	메뉴 버튼·키패드 모두 꺼짐
대기 줄이 없을 때	[Enter] 꺼짐
주문 내역 행을 고르지 않았을 때	[선택 취소]·[수량 변경]·[+][−] 꺼짐
품절 메뉴	그 메뉴 버튼 꺼짐

라. 경고창·안내창

조건	문구 (예시)
수량 0·빈 값	"수량은 1 이상이어야 합니다."
재고보다 많음	"라면은 재고가 3 개 남았습니다."
전체 취소 확인	"T3 의 주문 전체를 취소할까요?"

3.2 키오스크 음식·음료 주문 [키오스크]



[그림 ⑥-2] 키오스크 음식·음료 주문 — 색 상자가 P 영역

가. 화면 영역 구성

영역	컴포넌트	설명
P1	분류 탭(JToggleButton 4 개)	전체·음료·간식·식사
P2	MenuCard(직접 만든 JPanel) 여러 장, GridLayout(0, 4) + JScrollPane	품절 카드는 흐리게 + 클릭 안 됨
P3	장바구니 JList(셀 렌더러로 이름·단가·수량 표시) + [−][+][×], JLabel(총 주문 금액), JButton [주문하기]	장바구니는 ArrayList 에만 보관 (DB 아님)

나. 이벤트별 동작

이벤트 (리스너)	동작	DB
분류 탭	카드 다시 만들기	R menu
카드 클릭 (MouseListener)	장바구니에 담기(같은 메뉴는 수량만 +1)	—
[−][+][×]	장바구니 수량 변경·빼기, 총액 다시 계산	—
[주문하기]	주문 확인 팝업 열기	—

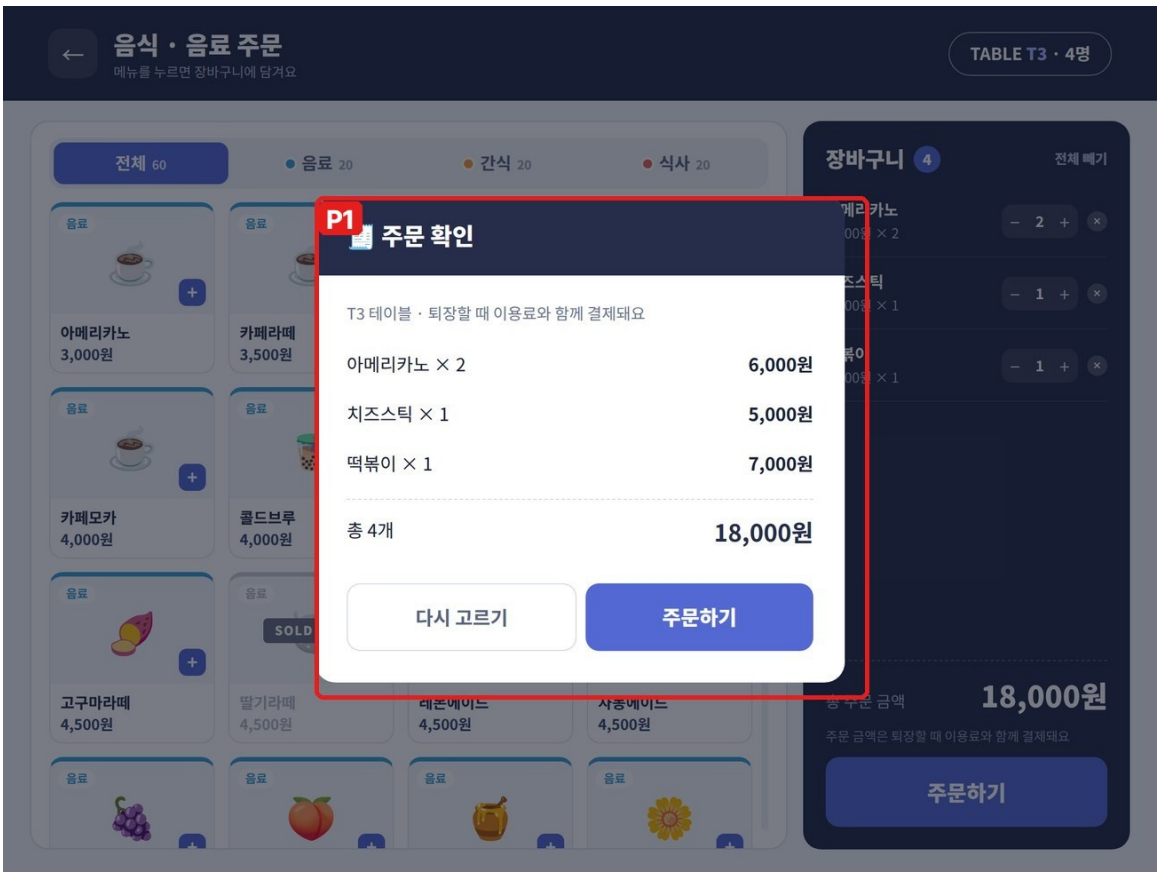
다. 버튼 켜기·끄기 규칙

상황	규칙
장바구니가 비었을 때	[주문하기] 꺼짐
품절 카드	클릭해도 담기지 않음

라. 경고창·안내창

조건	문구 (예시)
재고보다 많이 담음	"남은 수량은 3 개예요."

3.3 키오스크 주문 확인 팝업 [키오스크]



[그림 ⑥-3] 키오스크 주문 확인 팝업 — 색 상자가 P 영역

가. 화면 영역 구성

영역	컴포넌트	설명
P1	JDialog — JLabel 목록(메뉴 × 수량, 금액), 총액, [다시 고르기] [주문하기]	

나. 이벤트별 동작

이벤트 (리스너)	동작	DB
[주문하기]	주문 저장 트랜잭션, 경로 '키오스크', 상태 '접수' → 성공하면 장바구니 비우기	C orders, order_detail

라. 경고창·안내창

조건	문구 (예시)
주문 사이에 품절됨 (rollback)	"그 사이 품절된 메뉴가 있어요: 츄러스"
완료	"주문이 접수됐어요. 잠시만 기다려 주세요."

3.4 이용 현황 [키오스크]



[그림 ⑥-4] 이용 현황 — 색 상자가 P 영역

가. 화면 영역 구성

영역	컴포넌트	설명
P1	JLabel(이용 시간, 예상 이용 요금, 이용 요금, 음식 금액)	
P2	JTable 또는 JLabel 목록(내 주문 내역)	
P3	JLabel(마지막 갱신 시각) + JButton [돌아가기]	

나. 이벤트별 동작

이벤트 (리스너)	동작	DB
화면 열림 / 30 초마다 (Thread)	③ KioskFrame 의 테이블 → ⑤ TableUsageDAO.findActiveByTable() → 이용 시간, ⑤ FeeCalculator 로 이용 요금, order_detail 합계로 음식 금액, 예상 요금 = 둘의 합	R table_usage, order_detail
[돌아가기]	KioskFrame.show("메인"), 스레드 멈춤	—

라. 경고창·안내창

조건	문구 (예시)
이용 중 아님	"입장 등록 후 이용할 수 있어요."

3.5 키오스크 진입 흐름 (참고) [키오스크]



[그림 ⑥-5] 키오스크 진입 흐름 (참고) — 색 상자가 P 영역

가. 화면 영역 구성

영역	컴포넌트	설명
P1	1 단계 테이블 지정	③ — 설치할 때 직원이 한 번만
P2	2 단계 서비스 선택	③ 키오스크 메인
P3	3 단계 음식·음료 주문	⑥ 내 화면

4. DB 설계

DB 표에서 C = 등록(INSERT), R = 조회(SELECT), U = 수정(UPDATE), D = 삭제(DELETE)입니다. 테이블은 「공통 개발 가이드」의 SQL 로 팀 전원이 똑같이 만듭니다.

4.1 내가 담당하는 테이블

테이블 : orders (주문) ☑️ 담당

필드명	필드 자료형	PK / FK	필드 설명
order_id	INT (AUTO_INCREMENT)	PK	주문 번호
usage_id	INT	FK → table_usage.usage_id	어느 테이블 이용 건의 주문인지 (후불제)
staff_id	INT	FK → staff.staff_id	주문을 받은 직원 (키오스크 주문은 NULL)
channel	VARCHAR(10)		주문 경로 (POS / 키오스크)
order_time	DATETIME		주문 일시
total_amount	INT		주문 금액 합계 (= 주문 상세 소계의 합)
status	VARCHAR(10)		상태 (접수 / 확인). 키오스크 주문은 '접수'로 들어오고 직원이 확인하면 '확인'

테이블 : order_detail (주문 상세) ☑️ 담당

필드명	필드 자료형	PK / FK	필드 설명
order_detail_id	INT (AUTO_INCREMENT)	PK	주문 상세 번호
order_id	INT	FK → orders.order_id	어느 주문의 상세인지
menu_id	INT	FK → menu.menu_id	주문한 메뉴
qty	INT		주문 수량
unit_price	INT		주문 당시 단가 (menu.price 에서 불러옴)
subtotal	INT		소계 (qty × unit_price)

4.2 참조하는 다른 팀원의 테이블

굵게 표시한 필드는 **변경**까지 하는 필드입니다. 다른 팀원의 테이블은 직접 UPDATE 하지 않고 담당자의 DAO 메서드(5 장)를 부릅니다.

참조 테이블 (담당)	내가 쓰는 필드	쓰는 방법
menu (④ 박준영)	menu_id, name, category, price, **stock**, **sold_out**	조회 · 조회·변경
table_usage (⑤ 서윤채)	usage_id, table_no, people, start_time, status	조회

4.3 트랜잭션·처리 순서

주문 저장 (POS [Enter], 키오스크 [주문하기])

1. 자동 커밋 끄기
2. orders 1 줄 INSERT(이용 건, 직원 또는 NULL, 경로, 일시, 합계, 상태) → 생성된 order_id 받기
3. 메뉴마다 order_detail INSERT(단가는 menu.price 에서 불러옴, 소계 = 수량 × 단가)
4. 메뉴마다 ④ MenuDAO.decreaseStock(같은 Connection) — 재고가 부족하면 예외
5. commit

→ 재고 부족이나 오류가 나면 rollback → 주문 일부만 저장되거나 재고만 줄어드는 일이 없다.

주문 취소 ([선택 취소], [전체 취소])

1. 자동 커밋 끄기
2. order_detail DELETE
3. ④ MenuDAO.increaseStock(같은 Connection)으로 재고 되돌리기
4. 상세가 하나도 없는 orders 는 DELETE, 남아 있으면 total_amount 다시 계산 UPDATE
5. commit

→ 실패하면 rollback.

5. 클래스 설계

패키지 이름과 규칙은 「공통 개발 가이드」 3 장을 따릅니다. 아래 메서드 이름은 팀원끼리 서로 부르는 약속이므로 **굵은 메서드는 이름을 바꾸지 마세요.**

클래스	종류	하는 일	주요 메서드
OrderPanel	JPanel	손님 주문받기 화면	loadOrders(usage), buildMenuButtons(분류), addPending(메뉴), submit(), cancelSelected(), cancelAll()
MenuButtonListener	ActionListener 구현 클래스	모든 메뉴 버튼이 공유하는 리스너	actionPerformed(e) → e.getActionCommand()로 메뉴 번호 구분
KioskOrderPanel / MenuCard / OrderConfirmDialog	JPanel / JPanel / JDialog	키오스크 주문 / 메뉴 카드 / 확인 팝업	buildCards(분류), addToCart(메뉴), recalc() / OrderConfirmDialog(부모, 장바구니)
KioskUsagePanel	JPanel	이용 현황	refresh() (Thread), start(), stop()
OrderDAO	DAO	orders·order_detail SQL	create(헤더, 상세 목록) (트랜잭션), cancelDetail, cancelAll, updateQty, findByUsage, **findDetailsByUsage(usageId)**, countNewKiosk, confirmKiosk, **hasMenuOrder(menuId)**
OrderDTO / OrderDetailDTO / CartItem	DTO	테이블 한 줄, 장바구니 한 줄	필드 + getter/setter

다른 팀원과 맞물리는 부분

내가 제공하는 것 (다른 팀원이 부름)

메서드 / 화면	하는 일	쓰는 사람
OrderDAO.findDetailsByUsage(usageId)	이용 건의 주문 상세 전체 (메뉴 이름 포함)	② 결제 팝업
OrderDAO.hasMenuOrder(menuId)	그 메뉴의 주문 기록이 있는지	④ 메뉴 삭제
KioskOrderPanel, KioskUsagePanel	키오스크 화면 2 개	③ KioskFrame 에 등록

내가 쓰는 것 (다른 팀원이 만듦)

메서드 (담당)	쓰는 곳
MenuDAO.findByCategory / decreaseStock / increaseStock (④)	메뉴 목록, 주문·취소 시 재고
TableUsageDAO.findActiveAll / findActiveByTable (⑤)	테이블 콤보, 이용 현황
FeeCalculator (⑤)	이용 현황 요금
KioskFrame.show / getTableNo (③)	키오스크 화면 전환
Session.getStaffId() (①)	POS 주문 받은 직원
DBUtil (④)	DB 연결

6. 개발 일정 (9 주차 오리엔테이션 일정 기준)

주차	할 일
10 주차	주문받기·키오스크 주문·이용 현황 화면 배치, 메뉴 버튼 동적 생성과 공유 리스너, 장바구니(ArrayList)
11 주차	orders·order_detail INSERT·SELECT 연동, 저장 예시(881 번)와 같은 값이 들어가는지 확인
12~13 주차	주문·취소 트랜잭션(④ 재고 메서드 연결), 5 초 Timer·30 초 Thread, ③ KioskFrame 에 화면 연결, ② 에 findDetailsByUsage 제공
14 주차	키오스크 주문 → POS 알림 → 결제까지 통합 테스트, 결과보고서
15 주차	결과물 테스트

7. 완성 체크리스트 (“계획서대로 완성” 5 점)

수정본 계획서에 쓴 기능이 그대로 평가 기준입니다. 다 되면 ☐에 ✓ 하세요.

완료	화면	기능
☐	손님 주문받기	테이블 선택 (ItemListener) — 그 이용 건의 orders·order_detail JOIN 조회 → P2
☐	손님 주문받기	분류 선택 — ④ MenuDAO.findByCategory() → 메뉴 버튼 다시 만들기
☐	손님 주문받기	메뉴 버튼 (공유 ActionListener) — 대기 줄에 담기(같은 메뉴면 수량 +1)
☐	손님 주문받기	키패드 / 키보드 숫자 (KeyListener) — 고른 줄의 수량 입력
☐	손님 주문받기	[Enter] (주문) — 주문 저장 트랜잭션(개인 문서 4.3), 경로 ‘POS’, 상태 ‘확인’
☐	손님 주문받기	[+][−][수량 변경] — 저장된 줄이면 order_detail qty·subtotal UPDATE + orders.total_amount 다시 계산 + 재고 조정
☐	손님 주문받기	[선택 취소] / [전체 취소] — 확인창 → 주문 취소 트랜잭션
☐	손님 주문받기	5 초마다 (Timer) — 경로 ‘키오스크’ AND 상태 ‘접수’ 수 → P5 버튼 숫자. 누르면 목록 보여 주고 ‘확인’으로 UPDATE
☐	키오스크 음식·음료 주문	분류 탭 — 카드 다시 만들기
☐	키오스크 음식·음료 주문	카드 클릭 (MouseListener) — 장바구니에 담기(같은 메뉴는 수량만 +1)
☐	키오스크 음식·음료 주문	[−][+][✕] — 장바구니 수량 변경·빼기, 총액 다시 계산
☐	키오스크 음식·음료 주문	[주문하기] — 주문 확인 팝업 열기
☐	키오스크 주문 확인 팝업	[주문하기] — 주문 저장 트랜잭션, 경로 ‘키오스크’, 상태 ‘접수’ → 성공하면 장바구니 비우기
☐	이용 현황	화면 열림 / 30 초마다 (Thread) — ③ KioskFrame 의 테이블 → ⑤ TableUsageDAO.findActiveByTable() → 이용 시간, ⑤ FeeCalculator 로 이용 요금, order_detail 합계로 음식 금액, 예상 요금 = 둘의 합
☐	이용 현황	[돌아가기] — KioskFrame.show(“메인”), 스레드 멈춤
☐	트랜잭션	주문 저장 (POS [Enter], 키오스크 [주문하기]) : 실패 시 rollback 되는지 일부러 오류를 내서 확인
☐	트랜잭션	주문 취소 ([선택 취소], [전체 취소]) : 실패 시 rollback 되는지 일부러 오류를 내서 확인

8. 예상 질문과 답 (이해도 평가 10 점 대비)

교수님이 "왜 이렇게 만들었나요?"라고 물으면 아래 답을 **내 말로** 설명할 수 있어야 합니다. 외우기보다 이유를 이해하세요.

Q1. 주문을 왜 주문·주문 상세 2 개 테이블로 나눴나요?

교수님 피드백. 한 번 주문에 메뉴가 여러 개일 수 있어서 주문 1 번은 orders 1 줄, 메뉴마다 order_detail 1 줄로 저장한다.

Q2. 단가를 order_detail 에 저장하는 이유는?

주문 당시 가격을 남기려고. 나중에 메뉴 가격이 바뀌어도 주문 금액이 바뀌지 않는다.

Q3. 메뉴 버튼이 많은데 리스너는 어떻게 붙였나요?

리스너 객체 하나를 모든 버튼에 등록하고, 버튼마다 setActionCommand(메뉴 번호)를 해 두어 눌린 버튼을 구분한다.

Q4. Timer 와 Thread 를 둘 다 쓴 이유는?

POS 알림은 화면 안의 짧은 작업이라 Swing Timer(이벤트 스레드에서 실행)를, 이용 현황은 DB 조회와 계산을 따로 돌리려고 Thread 를 쓰고 화면 변경만 invokeLater 로 넘겼다.

Q5. 주문에 트랜잭션을 쓴 이유는?

주문 헤더, 상세 여러 줄, 메뉴 재고 감소가 함께 성공해야 해서. 재고 부족이면 전부 취소된다.

Q6. 키오스크 주문을 POS 가 어떻게 아나요?

키오스크 주문은 상태 '접수'로 저장되고, POS 가 5 초마다 '접수' 수를 세서 알린다. 직원이 확인하면 '확인'으로 바꾼다.

Q7. 장바구니는 왜 DB 에 저장하지 않나요?

주문하기 전까지는 손님이 마음대로 바꾸는 임시 데이터라서 ArrayList 에만 두고, [주문하기] 때 한 번에 저장한다.

9. 결과물 테스트 시연 순서

1. POS 주문받기: 테이블 선택 → 분류 → 메뉴 버튼 여러 번(같은 메뉴 수량 합치기) → 키패드로 수량 → Enter 로 주문 저장
2. 품절 메뉴 버튼이 꺼진 것, 재고보다 많이 주문 시 경고
3. 키오스크: 카드로 장바구니 담기 → 수량 조절 → 주문 확인 팝업 → 주문하기
4. POS 에서 5 초 안에 키오스크 주문 숫자가 뜨는 것 → 확인
5. 키오스크 이용 현황: 이용 시간·예상 요금, 30 초 갱신
6. 선택 취소 → 재고가 되돌아오는 것

시연 전에 「공통 개발 가이드」의 예시 데이터로 DB 를 새로 만들어 두면, 시연 중 데이터가 꼬여도 다시 시작할 수 있습니다.